Sapphire Language Specification and Automation Manual

Current syntax and automation behavior as implemented in the interpreter.

This manual describes the current local implementation, tested behavior, and honest boundaries of Sapphire. It is intentionally detailed so the document remains readable and useful beyond a short summary.

Language model

Sapphire is implemented as a dynamically typed, tree-walk interpreter rather than as a native machine-code compiler. This means the language is intentionally easier to inspect and reason about while still being capable of practical local scripting. It is a good fit for prototypes, experimentation, and controlled automation rather than for large-scale production claims.

The semantics revolve around expressions, statements, functions, control flow, structs, arrays, maps, and process results. The language aims to strike a balance between usability and practical automation features, which is why the project includes both script execution and CLI inspection tooling.

Automation primitives

The automation layer is responsible for the day-to-day operations users expect from a local scripting environment. This includes process and system queries, notifications, clipboard actions, file manipulation, HTTP support when present, scheduling, and controlled command execution. The system does not try to hide these operations; instead, it presents them as explicit capabilities with documented boundaries.

This makes Sapphire useful for desktop automation and local AI-assisted workflows. The user can write a script that checks system state, reads or writes files, and either sends a notification or triggers a local action with clear control over execution.

AI, ML, and optional dependencies

The AI and ML modules provide prompt and classification helpers, plus local tensor and optimization utilities. These features operate only when the relevant dependencies, drivers, or services are installed and configured. This is one of the project's clearest design decisions: capabilities are real when the environment supports them, not assumed by declaration alone.

This means that the same source code can behave differently across machines. The user may have the needed libraries on one computer and not on another, which is why the project describes optional packages and hardware requirements in a matter-of-fact way.

Portability and usage

The runtime is fundamentally Python-based, which makes it portable and inspectable. The packaged Windows distribution adds convenience tools and installation steps, but the behavior still depends on the local interpreter, the installed dependencies, and the underlying OS. The project therefore encourages users to validate their environment rather than assume a universal runtime state.

This is a strength rather than a weakness: the language is flexible enough to be useful in real machines and real developer setups, while the project keeps the documentation honest about what is and is not guaranteed.

Version 1.0.7. This document is intentionally longer than a brief summary to provide context, operational detail, and practical caveats for local use. Values and capabilities depend on the machine, software environment, and installed dependencies.